МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ
ПО ТЕХНИЧЕСКОМУ ЗАДАНИЮ
Система управления автозаправочной станции

по дисциплине
«Информатика и программирование»

Студент
гр. БИН-25-2 _____ В.А. Усяева

Ассистент
преподавателя _____ М.В. Водяницкий

Владивосток 2026

Задания

Техническое задание - Система управления автозаправочной станцией

Вы работаете в российской компании "НефтеСофт", разрабатывающей программное обеспечение для автоматизации автозаправочных станций.

Вам поручено создать консольный прототип системы управления АЗС, который используется операторами и техническим персоналом станции

Система должна учитывать реальные процессы работы заправки: продажи топлива, контроль запасов, обслуживание цистерн, аварийные ситуации и ведение статистики

1. Общая идея программы

Программа представляет собой консольную систему управления автозаправочной станцией, которая позволяет:

обслуживать клиентов (касса)

контролировать запасы топлива

управлять цистернами и колонками

оформлять пополнение топлива

вести историю операций и статистику

обрабатывать аварийные ситуации

Программа работает в виде меню с выбором действий и функционирует в непрерывном цикле до выхода пользователя

2. Топливо и цистерны

2.1 Типы топлива

На заправке используются следующие виды топлива:

АИ-92

АИ-95

АИ-98

ДТ (дизельное топливо)

2.2 Цистерны

Для одного типа топлива может существовать несколько подземных цистерн

Каждая цистерна имеет:

тип топлива

максимальный объем

текущий уровень топлива

состояние (включена / отключена)

минимальный допустимый уровень

Важно:

Если уровень топлива в цистерне падает ниже минимального порога:

цистерна автоматически отключается

из нее нельзя отпускать топливо

После пополнения:

цистерна не включается автоматически (после пополнения топливом)

включение производится вручную через меню

3. Колонки

3.1 Общая схема

На заправке есть несколько колонок

Каждая колонка:

поддерживает несколько типов топлива

каждый тип топлива (пистолет) подключен к конкретной цистерне

Не все колонки подключены ко всем цистернам

3.2 Схема заправки

Цистерны:

АИ-95 №1 -> колонки 1 - 4

АИ-95 №2 -> колонки 5 - 8

АИ-92 -> колонки 1 - 6

АИ-98 -> колонки 3 - 6

ДТ -> колонки 3 - 8

Всего колонок: 8

Каждая колонка имеет 2 - 3 пистолета

4. Главное меню программы

В главном меню пользователь может выбрать одно из действий:

```text

1) Обслужить клиента (касса)

2) Проверить состояние цистерн

3) Оформить пополнение топлива

4) Баланс и статистика

5) История операций
```

6) Перекачка топлива между цистернами

7) Включение / отключение цистерн

8) Состояние колонок

9) EMERGENCY - аварийная ситуация

0) Выход

```

5. Функциональные требования

5.1 Обслуживание клиента (касса)

Система должна:

принять запрос:

номер колонки

тип топлива

количество литров

проверить:

доступность колонки

состояние цистерны

достаточность топлива

рассчитать стоимость

запросить подтверждение оплаты

при подтверждении:

списать топливо

зафиксировать транзакцию

при нехватке топлива:

сообщить об ошибке

не допустить оплату

5.2 Проверка состояния цистерн

Вывод информации:

тип топлива

текущий объем

максимальный объем

состояние (включена / отключена)

предупреждения о низком уровне

5.3 Пополнение топлива

При прибытии бензовоза:
```

выбрать тип топлива

выбрать конкретную цистерну

указать количество литров

проверить, не превышает ли объем максимум

зафиксировать операцию в истории

5.4 Баланс и статистика

Система должна отображать:

общий доход

количество обслуженных автомобилей

сколько раз покупали каждый тип топлива

сколько литров каждого топлива продано

доход по каждому виду топлива

5.5 История операций

Вывод последних операций:

продажи топлива

пополнение цистерн

перекачка топлива

отключения и включения цистерн

аварийные события

5.6 Перекачка топлива

Перекачка возможна только между цистернами одного типа топлива

Указывается:

- источник
- приемник
- объем

Проверяется:

достаточность топлива

вместимость целевой цистерны

5.7 Управление цистернами

Пользователь может:

вручную отключать цистерны

вручную включать цистерны (если уровень топлива достаточен)

5.8 Состояние колонок

Выводится информация:

номер колонки

какие типы топлива доступны

к каким цистернам они подключены

какие пистолеты временно не работают

5.9 Аварийная ситуация (Emergency)

При подтверждении аварии:

все цистерны блокируются

заправка прекращает работу

фиксируется аварийное событие

имитируется вызов аварийных служб

Выход из аварийного режима возможен только вручную.

При выходе из аварийного режима цистерны автоматически не разблокируются.

6. Хранение данных

Все данные обязательно сохраняются в файлах (рекомендуется JSON):

состояние цистерн

схема колонок

баланс и статистика

история операций

Перезапуск программы не должен сбрасывать состояние заправки.

Содержание

1 Выполнение задания 3
1.1 Часть кода 1 3
1.2 Часть кода 2 3
1.3 Часть кода 3 4
1.4 Часть кода 4 4
1.5 Часть кода 5 5
1.6 Часть кода 6 6
1.7 Часть кода 7 6
1.8 Часть кода 8 7
1.9 Часть кода 9 8
1.10 Часть кода 10 9
1.11 Часть кода 11 9
1.12 Часть кода 12 10
1.13 Часть кода 13 11

1 Выполнение задания

1.1 Часть кода 1

Код представлен на рисунке 1.

```
import json, datetime, os

FUEL_TYPES = ["АИ-92", "АИ-95", "АИ-98", "ДТ"]
FUEL_PRICES = {"АИ-92": 48.50, "АИ-95": 58.30, "АИ-98": 66.80, "ДТ": 56.20}
DATA_FILE = "azs_data.json"
```

Рисунок 1 — Листинг программы 1

Модуль json создан для сохранения и загрузки данных в файл. Модуль datetime создан для временных меток операций. Модуль os создан для проверки существования файла данных. Определяется 4 вида топлива и их фиксированная стоимость. Задаётся имя файла для хранения данных.

1.2 Часть кода 2

Код представлен на рисунке 2.

```
class Cistern:
    def __init__(self, id, fuel_type, max_volume, current_volume, min_level=1000, is_active=
True):
        self.id, self.fuel_type, self.max_volume, self.current_volume = id, fuel_type, max_volume,
current_volume
        self.min_level, self.is_active, self.is_emergency_blocked = min_level, is_active, False

    def can_dispense(self, amount):
        return (self.is_active and not self.is_emergency_blocked and
                self.current_volume >= amount and self.current_volume - amount >= self.min_level)

    def dispense(self, amount):
        if self.can_dispense(amount):
            self.current_volume -= amount
            if self.current_volume < self.min_level:
                self.is_active = False
            return True
        return False

    def refuel(self, amount):
        if self.current_volume + amount <= self.max_volume:
            self.current_volume += amount
```

```
        return True
    return False
```

Рисунок 2 — Листинг программы 2

Класс Cistern представляет собой модель цистерны для хранения топлива на автозаправочной станции. Это фундаментальный компонент системы, который инкапсулирует всю логику работы с топливным резервуаром. Id уникальный идентификатор цистерны, fuel_type тип хранимого топлива, max_volume максимальная ёмкость цистерны в литрах, cyrrent_volume текущий объём топлива в цистерне, min_level минимальный допустимый уровень остатка в цистерне, is_active флаг активности цистерны для продаж, is_emergency_blocked блокировка при аварии. Can_dispense(amount) проверяет можно ли продать для этого должны выполняться условия: цистерна активна, не заблокирована аварийно, достаточно топлива для продажи, после продажи останется минимум литров. Refuel(amount) проверяет не превысит ли пополнение max_volume, если не превышает, увеличивает current_volume.

1.3 Часто кода 3

Код представлен на рисунке 3.

```
class Column:
  def __init__(self, column_id, pumps_config):
    self.column_id = column_id
    self.pumps = []
    for i, (fuel_type, cistern_id) in enumerate(pumps_config, 1):
      self.pumps.append(
        {"pump_id": f"P{column_id}-{i}", "fuel_type": fuel_type, "connected_cistern_id":
cistern_id})
```

Рисунок 3 — Листинг 3

Класс Column моделирует заправочную колонку на АЗС, которая содержит несколько пистолетов для разных типов топлива. Column_id номер колонки, pumps_config список конфигураций пистолетов, enumerate(pumps_config, 1) нумерация пистолетов с 1, f"P{column_id}-{i} создает ID, self.column_id номер колонки, self.pumps список словарей с информацией о списке. Класс создаёт структуру данных, которая связывает физические пистолеты на колонке с логическими цистернами в системе.

1.4 Часть кода 4

Код представлен на рисунке 4.

```
class AZSManager:
    def __init__(self):
        self.cisterns, self.columns, self.transactions, self.operations = {}, [], [], []
        self.statistics = {"total_cars": 0, "total_income": 0.0, "fuel_sold": {ft: 0.0 for ft in
FUEL_TYPES},
                           "fuel_income": {ft: 0.0 for ft in FUEL_TYPES}}
        self.is_emergency, self.next_ids = False, {"transaction": 1, "operation": 1}
        self._init_data()
        self.load_data()
```

Рисунок 4 — Листинг программы 4

Конструктор класса AZSManager создает центральный управляющий объект всей системы АЗС, инициализируя все структуры данных. Self.cistern словарь, self.columns список объектов Column, self.transactions история продаж, self.operations аудит. Self.statistics общее число обслуженных авто и доход, количество проданного по типам топлива и доход по каждому типу. Self._init_data() создание начальных данных, self.load_data() загрузка сохранённых данных.

1.5 Часть кода 5

Код представлен на рисунке 5.

```
def _init_data(self):
    cisterns_data = [("АИ-92 №1", "АИ-92", 20000, 12400), ("АИ-95 №1", "АИ-95", 20000,
9800),
                     ("АИ-95 №2", "АИ-95", 20000, 1200), ("АИ-98 №1", "АИ-98", 15000, 10000),
                     ("ДТ №1", "ДТ", 25000, 15600)]
    for cid, ft, mv, cv in cisterns_data:
        self.cisterns[cid] = Cistern(cid, ft, mv, cv, 500)
 if self.cisterns[cid].current_volume < 500:
        self.cisterns[cid].is_active = False

column_config = {1: [("АИ-92", "АИ-92 №1"), ("АИ-95", "АИ-95 №1")],
                 2: [("АИ-92", "АИ-92 №1"), ("АИ-95", "АИ-95 №1")],
                 3: [("АИ-92", "АИ-92 №1"), ("АИ-95", "АИ-95 №1"), ("АИ-98", "АИ-98 №1"),
("ДТ", "ДТ №1")],
                 4: [("АИ-92", "АИ-92 №1"), ("АИ-95", "АИ-95 №1"), ("АИ-98", "АИ-98 №1")],
                 5: [("АИ-92", "АИ-92 №1"), ("АИ-95", "АИ-95 №2"), ("АИ-98", "АИ-98 №1"),
("ДТ", "ДТ №1")],
                 6: [("АИ-92", "АИ-92 №1"), ("АИ-95", "АИ-95 №2"), ("АИ-98", "АИ-98 №1"),
("ДТ", "ДТ №1")],
                 7: [("АИ-95", "АИ-95 №2"), ("ДТ", "ДТ №1")], 8: [("АИ-95", "АИ-95 №2"),
 ("ДТ", "ДТ №1")]}
```

```
for col_id, config in column_config.items():
    self.columns.append(Column(col_id, config))
```

Рисунок 5 — Листинг программы 5

Метод создаёт начальную конфигурацию АЗС — цистерны с топливом и заправочные цистерны с пистолетами. Создана цистерна с минимальным уровнем 500л., она деактивируется если текущий объём меньше 500л.

1.6 Часть кода 6

Код представлен на рисунке 6.

```
def save_data(self):
    data = {"cisterns": [{**c.__dict__} for c in self.cisterns.values()],
            "transactions": self.transactions, "operations": self.operations,
            "statistics": self.statistics, "is_emergency": self.is_emergency,
            "next_ids": self.next_ids}
    with open(DATA_FILE, 'w', encoding='utf-8') as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def load_data(self):
    if not os.path.exists(DATA_FILE): return
    with open(DATA_FILE, 'r', encoding='utf-8') as f:
        data = json.load(f)
    self.cisterns = {c['id']: Cistern(**c) for c in data.get("cisterns", [])}
    self.transactions, self.operations = data.get("transactions", []), data.get("operations", [])
    self.statistics, self.is_emergency = data.get("statistics", self.statistics), data.get
("is_emergency", False)
    self.next_ids = data.get("next_ids", self.next_ids)
```

Рисунок 6 — Листинг программы 6

Методы save_data и load_data обеспечивают сохранение состояния АЗС в файл и его восстановление при следующем запуске программы. Метод save_data сохраняет в файл. 'W' файл открывается для записи, encoding='utf-8' для поддержки кириллицы, ensure_ascii=False сохраняет русские буквы, indent=2 форматирование с отступами. Метод load_data создан для загрузки из файла. Data.get("cisterns", []) берёт список цистерн или пустой список, Cistern(**c) создаёт объект Cistern из словаря с атрибутами.

1.7 Часть кода 7

Код представлен на рисунке 7.

```
def _add_op(self, op_type, details):
    self.operations.append(
        {"id": self.next_ids["operation"], "timestamp": datetime.datetime.now().strftime("%Y-%m-%d
%H:%M:%S"),
         "operation_type": op_type, "details": details})
    self.next_ids["operation"] += 1
```

Рисунок 7 — Листинг программы 7

Метод _add_op() добавляет записи в журнал операций системы АЗС. Это системный журнал, куда записываются все действия для последующего анализа. Op_type тип операции, details подробное описание операции, self.next_ids["operation"] текущий номер операции.

1.8 Часть кода 8

Код представлен на рисунке 8.

```
def serve_customer(self):
    if self.is_emergency: return print(" Аварийный режим!")
    print("\n=== ОБСЛУЖИВАНИЕ КЛИЕНТА ===")
    avail_cols = [c for c in self.columns if any(p["fuel_type"] in FUEL_TYPES for p in c.pumps)]
    if not avail_cols: return print("Нет доступных колонок")

    for i, col in enumerate(avail_cols, 1):
        print(f"{i}) Колонка {col.column_id}")
    try:
        col_choice = int(input("Выберите колонку: ")) - 1
        col = avail_cols[col_choice]

        avail_fuels = [p for p in col.pumps if self.cisterns[p["connected_cistern_id"]].is_active]
        for i, p in enumerate(avail_fuels, 1):
            print(f"{i}) {p['fuel_type']} ({FUEL_PRICES[p['fuel_type']]} ₽/л)")

        fuel_choice = int(input("Выберите топливо: ")) - 1
        pump = avail_fuels[fuel_choice]
        cistern = self.cisterns[pump["connected_cistern_id"]]

        liters = float(input(f"Литры (до {cistern.current_volume} л): "))
        if liters > cistern.current_volume: return print("Недостаточно топлива")

        total = liters * FUEL_PRICES[pump["fuel_type"]]
        print(f"{liters} л × {FUEL_PRICES[pump['fuel_type']]} ₽ = {total:.2f} ₽")

        if input("Подтвердить? (y/n): ").lower() == 'y':
            if cistern.dispense(liters):
```

```
            self.transactions.append({"id": self.next_ids["transaction"],
                                      "timestamp": datetime.datetime.now().strftime("%Y-%m-%d %H:
%M:%S"),
                                      "column_id": col.column_id, "fuel_type": pump["fuel_type"],
                                      "liters": liters,
                                      "total_price": total, "cistern_id": cistern.id})
            self.statistics["total_cars"] += 1
            self.statistics["total_income"] += total
            self.statistics["fuel_sold"][pump["fuel_type"]] += liters
            self.statistics["fuel_income"][pump["fuel_type"]] += total
            self.next_ids["transaction"] += 1
            self._add_op("sale", f"Продажа {liters} л {pump['fuel_type']} за {total:.2f} ₽")
          print("Успешно")
        else:
          print("Ошибка")
    except:
      print("Ошибка ввода")
    input("Нажмите Enter...")
```

Рисунок 8 — Листинг программы 8

Основной метод для продажи топлива клиенту. Реализует полный процесс обслуживания от выбора колонки до записи транзакции. Если включён аварийный режим продажа отменяется. Фильтрует колонки, где есть хоть один тип топлива из FUEL_TYPES . Если нет доступных колонок выходит. Клиент выбирает номер колонки. Если нажать ENTER возвращает в главное меню.

1.9 Часть кода 9

Код для выполнения задания представлен на рисунке 9.

```
def check_cisterns(self):
    print("\n=== СОСТОЯНИЕ ЦИСТЕРН ===")
    for c in self.cisterns.values():
        status = "ВКЛ" if c.is_active else "ВЫКЛ"
        if c.is_emergency_blocked: status = "БЛОК"
        print(f"{c.id} | {c.fuel_type} | {c.current_volume}/{c.max_volume} л | {status}")
    input("Нажмите Enter...")
```

Рисунок 9 — Листинг программы 9

Метод показывает текущее состояние всех цистерн на АЗС в виде таблицы. Это инструмент мониторинга для оператора. Выводит заголовок для визуального разделения. C.id идентификатор колонки, c.fuel_type тип хранимого топлива, status текущий статус работы. Метод check_cisterns предоставляет оператору моментальный

снимок состояния всех топливных резервуаров, помогая принимать решения о пополнении запасов и выявлять проблемы.

1.10 Часть кода 10

Код представлен на рисунке 10.

```
def refuel_cistern(self):
    if self.is_emergency: return print("Аварийный режим!")
    print("\n=== ПОПОЛНЕНИЕ ===")
    for i, ft in enumerate(FUEL_TYPES, 1):
        print(f"{i}) {ft}")
    try:
        fuel_choice = int(input("Выберите топливо: ")) - 1
        ft = FUEL_TYPES[fuel_choice]
        cisterns = [c for c in self.cisterns.values() if c.fuel_type == ft]
        for i, c in enumerate(cisterns, 1):
            print(f"{i}) {c.id}: {c.current_volume}/{c.max_volume} л")

        c_choice = int(input("Выберите цистерну: ")) - 1
        c = cisterns[c_choice]
        liters = float(input(f"Литры (макс {c.max_volume - c.current_volume}): "))

        if c.refuel(liters):
            self._add_op("refuel", f"Пополнение {c.id} на {liters} л")
            print("Успешно")
        else:
            print("Ошибка")
    except:
        print("Ошибка ввода")
    input("Нажмите Enter...")
```

Рисунок 10 — Листинг программы 10

Метод позволяет пополнить запас топлива в цистернах АЗС. Это операция закупки/дозаправки для восстановления запасов. Фильтруются цистерны по выбранному виду топлива. Показывает максимальный объём топлива. Вызывает метод объекта цистерны refuel(), если успешно записывает операцию в журнал, если неудачно выводит ошибку. Метод refuel_cistern обеспечивает процесс восстановления топливных запасов с контролем переполнения и полным логированием операций закупки.

1.11 Часть задания 11

Код представлен на рисунке 11.

```
def show_stats(self):
    print("\n=== СТАТИСТИКА ===")
    print(f"Автомобилей: {self.statistics['total_cars']}")
    print(f"Доход: {self.statistics['total_income']:.2f} ₽")
    for ft in FUEL_TYPES
        liters = self.statistics['fuel_sold'][ft]
        income = self.statistics['fuel_income'][ft]
        if liters > 0:
            print(f"{ft}: {liters:.1f} л, {income:.2f} ₽")
    input("Нажмите Enter...")
```

Рисунок 11 — Листинг программы 11

Метод show_stats отображает статистику работы АЗС - общие показатели и детализацию по типам топлива. Выводит заголовок для визуального разделения. Показывает количество обслуженных машин и общий доход.

1.12 Часть кода 12

Код представлен на рисунке 12.

```
def emergency_mode(self):
    print("\nАВАРИЙНЫЙ РЕЖИМ")
    confirm = input("АКТИВИРОВАТЬ? (y/n): ").lower()
    if confirm == 'y':
        self.is_emergency = True
        for c in self.cisterns.values():
            c.is_emergency_blocked = True
        self._add_op("emergency", "Активация аварийного режима")
        print(" Аварийный режим активирован")
    else:
        self.is_emergency = False
        for c in self.cisterns.values():
            c.is_emergency_blocked = False
        self._add_op("emergency", "Отключение аварийного режима")
        print(" Аварийный режим отключен")
    input("Нажмите Enter...")
```

Рисунок 12 — Листинг программы 12

Метод управляет аварийным режимом работы АЗС - блокирует или разблокирует все операции с топливом в случае чрезвычайной ситуации. Показывает аварийный режим, запрашивает разрешение. Метод emergency_mode реализует критически важную функцию безопасности, позволяя мгновенно остановить все операции с топливом в случае аварии или технической необходимости, с полным логированием и контролем доступа.

1.13 Часть кода 13

Код представлен на рисунке 13

```
def main():
    azs = AZSManager()
    while True:
        print("\n" + "=" * 50)
        print("АЗС УПРАВЛЕНИЕ")
        print("=" * 50)
        print("АЗС УПРАВЛЕНИЕ")
        print("=" * 50)
        print("1) Обслужить клиента")
        print("2) Проверить цистерны")
        print("3) Пополнить цистерну")
        print("4) Статистика")
        print("9) Аварийный режим")
        print("0) Выход")

        try:
            choice = int(input("Выбор: "))
            if choice == 0:
                azs.save_data()
                break
            elif choice == 1:
                azs.serve_customer()
            elif choice == 2:
                azs.check_cisterns()
            elif choice == 3:
                azs.refuel_cistern()
            elif choice == 4:
                azs.show_stats()
            elif choice == 9:
                azs.emergency_mode()
        except:
            print("Ошибка")

if __name__ == "__main__":
```

Рисунок 13 — Листинг программы 13

Функция main является главным циклом управления программой - создает систему АЗС и предоставляет интерфейс для взаимодействия с ней через консольное меню. Программа работает пока пользователь не выберет выйти. После каждой операции возвращает в меню.